Capstone 1 – Policy Clause Finder: understanding guide

A study guide to what was built, how it works and why. Read it before the trainer walkthrough. It is for your own understanding and is not part of the submission (it lives in `understanding/`, which is not pushed to GitLab). The code is in `capstone-1/`.

---

1. The capstone in one page

What the brief asks for. Bank operations analysts lose time searching policy documents for the one clause that answers a customer. Keyword search fails when the customer's words differ from the policy's words: the customer says "*I cannot pay my EMI*" and the policy says "*repayment default*". Build a small **semantic search** tool that:

1. turns each policy clause and the question into **vectors** (lists of numbers),
2. ranks the clauses by **cosine similarity** and returns the **top 3 with scores**,
3. checks every quote it shows **exists word for word** in the source clause,
4. says **"not found"** when even the best score is too low.

What was built. A command-line tool in Python with 19 synthetic clauses, a banking "concept" embedder, cosine written by hand, a quote verifier, a threshold of 0.40, an evaluation script and 23 unit tests.

Status.

Item	Status
Brief steps 1–7 (set-up, clause store, embed, rank, verify, test, document)	Done
Acceptance checklist (5 items)	All 5 met
Runs from a fresh copy in a new virtual environment	Checked
Pushed to your GitHub holding repo (`genaiops_training/capstone-1`)	Done
Push to **GitLab** and open a **merge request** for the trainer	**You**: needs the GitLab URL from the trainer
8-minute walkthrough with the trainer	**You**: script in `capstone-1/understanding/WALKTHROUGH.md`

---

2. The ideas you need, in plain words

2.1 A vector (an "embedding")

A vector is just a list of numbers. Here each position means something. Position 1 is "how much is this text about *repayment trouble*?", position 2 is "how much is it about *loans*?", and so on. Turning text into such a list is called **embedding** the text.

Two texts about the same thing get similar lists **even if they share no words**. That is the whole trick that beats keyword search.

2.2 The four maths functions (in `rank.py`)

For two vectors **a** and **b**:

Function	Formula	What it means
dot product	a1·b1 + a2·b2 + …	Multiply matching positions and add up. Big when both are high on the same positions
norm (length)	√(dot(a, a))	How long the arrow is (Pythagoras in many dimensions)
cosine similarity	dot(a, b) / (norm(a) · norm(b))	The angle between the arrows. 1 = same direction, 0 = nothing in common. **Ignores length**
Euclidean distance	$\sqrt{((a1-b1)^2 + (a2-b2)^2 + \ldots)}$	Straight-line gap between the arrow tips. Lower = closer. **Affected by length**

2.3 Why cosine and not Euclidean

A policy clause is long, so its vector is long. A question is short, so its vector is short. Euclidean distance mostly measures that difference in length, so it always picks the **shortest clauses** whatever the question. Measured on the 8 test questions: **Euclidean 0/8 correct, cosine 8/8**.

Cosine only looks at the *direction*, meaning which topics and in what mix. A short question and a long clause about the same topic point the same way.

If every vector is first scaled to length 1 ("unit vectors"), Euclidean ranks exactly like cosine (also 8/8), because then $distance^2 = 2 - 2 \times cosine$. That is the identity from Day 3.

2.4 The threshold ("not found")

Cosine always produces *some* top 3, even for "What time does the canteen open?". So the tool needs a minimum score: below **0.40** it answers **NOT FOUND** and sends the analyst to the policy team. A wrong answer is worse than no answer.

2.5 Verification

The tool shows a **quote**, the one sentence of the clause that best matches the question. Before showing it, the code checks that the quote appears **character for character** in the clause store. Today the code cuts the quote from the clause itself, so it always passes. The check is a guard: if a later change, or a language model, ever rewords or invents a quote, it is caught and withheld. This is the Day 2 lesson: *verify model output against the source*.

---

3. One question, step by step (real numbers)

Question: **"Customer says they cannot pay their EMI this month. What happens to the loan?"**

Step 1 – clean the words (`embed.tokenize`). The text is lower-cased, "can't" becomes "cannot", and each word is lightly stemmed ("happens" → "happen", "charged" → "charg"), giving: `customer say they cannot pay their emi thi month what happen to the loan`

Step 2 – find concepts (`embed.ConceptEmbedder`). The tool reads the longest known phrase first:

Found	Concept dimension
"cannot pay"	repayment_trouble
"emi"	loan
"loan"	loan
"month", "happen" (unknown words)	small nudges (0.3) on two hashed dimensions

Found	Concept dimension
"customer", "they", "what", "the" …	ignored (stop words)

The question vector (only the non-zero positions): **repayment_trouble 1, loan 2, hash6 0.3, hash24 0.3**.

Step 3 – the clause vector. Clause **LN-01 "Missed instalment and repayment default"** was embedded once at start-up (title + text): **repayment_trouble 8, loan 7, time_limit 4, early_settlement 1**, plus small hashed values. Its words are "in arrears", "not paid", "unpaid", "repayment default", "instalment", "loan", "borrower" and so on. They share almost no words with the question, but they land on the **same two concepts**.

Step 4 – cosine, by hand (`rank.cosine_similarity`):

- dot = 1×8 + 2×7 + 0.3×0.6 + 0.3×0.3 = **22.27**
- norm(question) = $\sqrt{(1^2 + 2^2 + 0.3^2 + 0.3^2)}$ = **2.276**
- norm(clause) = **11.535**
- cosine = 22.27 / (2.276 × 11.535) = **0.848**

numpy and scikit-learn give 0.848265 as well (`results/crosscheck.txt`).

Step 5 – rank all 19 clauses. Top 3: **LN-01 0.848**, LN-04 0.798 (late payment charge on loans), LN-03 0.466. 0.848 is above 0.40, so the answer is "found".

Step 6 – pick and verify the quote (`verify.py`). Each sentence of LN-01 is embedded and the closest one is chosen: "*If any amount remains unpaid for 90 days, the loan is classified as a repayment default and the default is reported to the credit reference agencies.*" It is found verbatim in LN-01, so it is shown as `Quote [verified in LN-01]`.

Step 7 – print, ending with the human-review line: read the full clause before replying.

Note the Euclidean distance for LN-01 is 9.68, larger than for many short clauses. That is why Euclidean on raw vectors gets it wrong.

---

4. File-by-file tour

File	What it does	Key parts to understand
`finder.py`	The CLI and the `ClauseFinder` class that joins everything	`load_clauses()` validates the store (fields present, ids unique); `search()` embeds, ranks, picks and verifies quotes, applies the threshold; `THRESHOLD = 0.40`; options `--explain`, `--json`, `--top`, `--threshold`, `--interactive`
`embed.py`	Text → vector	`CONCEPTS`: the 19 concepts and their words/phrases; `stem()`, `tokenize()`; greedy longest-phrase matching (`_matches`); 32 hashed dimensions for unknown words; `KeywordEmbedder`, the no-synonyms baseline
`rank.py`	The maths, by hand	`dot`, `norm`, `cosine_similarity` (error on a zero vector), `euclidean_distance` (error on different lengths), `rank()` (top-k, stable ties, empty result if nothing recognised)
`verify.py`	Quote selection and checking	`split_sentences()`, `best_snippet()`, `verify_quote()` (exact substring)
`crosscheck.py`	Proves the hand maths is right	Compares with numpy and scikit-learn on two examples to 3 decimal places
`evaluate.py`	Honest test report	Runs 8 main + 6 stress + 2 out-of-scope questions; compares cosine, Euclidean, unit-Euclidean and keyword; writes `results/test_results.md`

File	What it does	Key parts to understand
`tests/test_clause_finder.py`	23 unit tests	Maths, embedder, verifier (altered quote rejected), clause store (size, near-duplicates), end to end, CLI
`data/clauses.json`	19 synthetic clauses	ID, category, title, text; `near_duplicate_of` marks 4 near-duplicate pairs
`data/test_questions.json`	Questions with expected answers	8 main, 6 stress, 2 out-of-scope, each with why it is tricky
`docs/architecture.png`	The diagram	Redraw with `python docs/make_diagram.py`
`README.md`	The submission document	Purpose, how to run, results, limitations, improvement, checklist

---

5. The data

19 clauses from a fictional bank: loans (LN-01 to 05), cards (CC-01 to 05), complaints (CP-01 to 03), KYC (KY-01 to 04), accounts (AC-01, 02).

Near-duplicates (the brief asks for at least 3). These are pairs on the same topic that differ in one key detail, built to make ranking hard:

Pair	Same topic	The difference
LN-04 / CC-03	late payment charge	loan 25.00 vs credit card 12.00
CC-02 / CC-04	card payment the customer disputes	not made by the customer (fraud) vs merchant didn't deliver
CP-01 / CP-03	complaint response time	standard 15 days vs vulnerable customers 10 days
KY-02 / KY-04	periodic KYC review	personal vs business customers

Questions. There are 8 main questions written in customer words (the brief's step 6) and 6 stress questions written afterwards with wording the lexicon doesn't know. There are also 2 off-topic questions that must come back "not found".

---

6. Results and what they mean

Test	Right first time	Meaning
8 main questions, the tool	**8/8**	Works on the target questions, including all the near-duplicate traps
8 main, keyword search	3/8	Shows the business problem: no synonyms, so it misses
8 main, Euclidean	0/8	Shows why cosine is used
8 main, unit vectors + Euclidean	8/8	Day 3 identity: on unit vectors both metrics agree
6 stress questions	**1/6** (6/6 in top 3)	The honest weakness: unknown synonyms
2 off-topic questions	2/2 "not found"	The threshold works

Why report the 1/6? The same person wrote the synonym list and the main questions, so 8/8 is flattering. The stress set was written afterwards and **nothing was tuned** after seeing it. The brief marks "Test results with honest counts, including failures". Example failure: "*My husband passed away… how quickly will you deal with my complaint?*" Only "complaint" is recognised ("passed away" is not linked to "bereavement"), so CP-02 comes first instead of CP-03.

Threshold choice. Off-topic questions score at most 0.255 and the main questions at least 0.725, so 0.40 sits in the gap. One stress question (S4, 0.269) falls below it and gets "not found". That is the safe way to fail.

---

7. Brief → where it was done

Brief step	Done in
1 Set up: folder, virtual environment, README, `.gitignore` with .env and venv	`capstone-1/`, `.venv/` (not committed), `README.md`, `.gitignore`
2 Clause store: 15–20 clauses, loans/cards/complaints/KYC, ≥3 near-duplicates	`data/clauses.json` (19 clauses, 4 pairs)
3 Embed: `embed.py`	`embed.py`
4 Rank: dot, norm, cosine by hand; top 3; library cross-check on two examples	`rank.py`, `crosscheck.py`
5 Verify: quote verbatim; "not found" below a documented threshold	`verify.py`, `THRESHOLD` in `finder.py`, README "Choosing the threshold"
6 Test: 8 questions with expected clause ID, right-first-time count	`data/test_questions.json`, `evaluate.py`, `results/test_results.md`
7 Document: purpose, how to run, results table, two limitations, one improvement	`README.md`
Push: code, README, test results, architecture diagram in `capstone-1`	GitHub done; GitLab is for you

Acceptance item	Evidence
Runs end to end on a fresh clone	Re-run in a fresh copy with a new venv: tests 23/23, evaluation identical
Top 3 show clause ID, score, verified quote	`results/live_run_1_normal.txt`
Hand cosine matches library to 3 dp	`results/crosscheck.txt` (identical to 6 dp)
8-question test table with honest results	`results/test_results.md`, README
Only synthetic data, no .env or keys	Fictional bank; no API used; key scan clean

---

8. Design decisions, and how to explain them

Why not a real embedding model? The brief allows "local embedding or the simple lab embedder". The real local model (fastembed, Day 2) cannot load on this machine because the Microsoft Visual C++ runtime is missing. The concept embedder also has two advantages: you can explain every number by eye, and it needs no downloads.

Why phrases, longest first? "lost my job" must mean hardship, not a lost card. Reading the longest known phrase first stops "lost" being counted on its own.

Why the hashed dimensions? Without them, two texts with the same concepts would be identical. Unknown words add a small, repeatable nudge (weight 0.3), so specific words still count a little.

Why embed title + text? The title often carries the key concept ("Dormant accounts").

Why verify a quote the code cut itself? It is a guard against future changes or a model producing the quote. A test proves an altered quote (2 days instead of 5, an extra space) is rejected.

Why print a human-review line? A similarity score means "same topic", not "this answers the question". Opposite statements can score high (Day 3, Task E). Q5 shows how close near-duplicates come: CP-01 0.951 vs CP-02 0.937.

---

9. How to run and demo

```
cd capstone-1
python -m venv .venv
.venv\Scripts\activate                          # Windows (macOS/Linux: source .venv/bin/activate)
pip install -r requirements.txt

python finder.py "Customer says they cannot pay their EMI this month. What happens to the loan?" --explain
python finder.py "I paid my credit card bill two days late, will I be charged anything?" --explain
python finder.py "What time does the staff canteen open on Fridays?"
python crosscheck.py
python evaluate.py
python -m unittest discover -s tests -v
```

Walkthrough order (about 8 minutes): problem → diagram → normal run (EMI) → tricky run (credit card late fee vs loan late charge) → not found (canteen) → results table and the stress-set failures → limitation and next step. The full script is in `understanding/WALKTHROUGH.md`.

---

10. Limitations and the next step

1. **Only as good as its synonym list.** New wording is missed: 1/6 on the stress set.
2. **No understanding of "not" or numbers.** Near-duplicates score close together, so the analyst must read the clause.

Next step: swap in a real local embedding model (fastembed `bge-small`, once the Visual C++ runtime is installed). Keep the ranker, verifier and threshold, re-run the same 14 questions and re-choose the threshold.

Glossary

Term	Meaning
Embedding	Text turned into a list of numbers that represents its meaning
Cosine similarity	Angle-based closeness of two vectors, from 0 to 1 here; ignores length
Euclidean distance	Straight-line distance between two vectors; affected by length
Unit vector	A vector scaled to length 1
Stemming	Cutting word endings so word forms match ("charged" → "charg")
Stop words	Very common words ignored ("the", "my", "what")
Near-duplicate	Two clauses on the same topic with a small but important difference
Threshold	Minimum score to count as found
Right first time	The expected clause is ranked number 1
Verbatim	Exactly the same characters